Specification-Driven Development: A Report

Oct 29, 2026 · @V

Executive summary

Specification-Driven Development (SDD) makes a written, version-controlled specification the primary artifact of engineering work, and treats code as something derived from it and checked against it. AI coding agents made this practical and necessary: when an agent can produce thousands of lines in minutes, the scarce skill is no longer typing code but stating precisely what should exist and how anyone will know it is right.

- **What it is.** Write the spec first, implement against it, then validate the result against it. The term is still loosely used; Thoughtworks' Birgitta Böckeler separates three depths: *spec-first*, *spec-anchored* and *spec-as-source*. Almost all real practice today is spec-first or spec-anchored.
- **Why the spec, not the prompt.** A prompt is a one-off instruction that vanishes with the chat session. A spec is durable, reviewable, diffable, shared across people and agents, and testable. The prompt shrinks to a pointer ("implement task 3 of this spec"), and the spec carries the intent.
- **What a good spec contains.** Six parts: intent (why), scope (what is in and out), constraints (what must hold), interfaces (how it connects to the world), acceptance criteria (what "done" means in testable terms) and test cases (concrete examples that prove it).
- **Where it goes wrong.** Over-specifying small changes, review overload from piles of markdown, specs that drift away from the code, and agents that treat specs as suggestions. SDD reduces risk; it does not remove the need to review and test the output.

The rest of this report defines SDD, builds the argument for the spec as source of truth, walks through each of the six components with examples, surveys the tooling (Spec Kit, Kiro, Tessl and others) and closes with risks and an adoption checklist.

What is specification-driven development?

SDD is a way of building software, usually with AI coding agents, in which you write an explicit specification first, implement against it, and then validate that the implementation matches it. Böckeler's working definition on martinfowler.com is that you write a spec before writing code with AI, and the spec becomes the source of truth for both the human and the AI. One practitioner glossary puts the same idea in a line: the spec, not a prompt, is what the agent implements from (Prefactor glossary).

The practice grew out of a reaction to "vibe coding", the style Andrej Karpathy named in February 2025, where you describe a goal to an assistant and accept whatever comes back (Allstacks). Vibe coding is fine for throwaway prototypes. It breaks down when the agent has to guess goals, constraints and domain rules you never told it. Thoughtworks counted SDD among the most important practices to emerge in 2025 (Thoughtworks).

What counts as a "spec"

Böckeler defines a spec as a structured, behavior-oriented artifact, or set of artifacts, written in natural language that describes software functionality and guides AI coding agents. She also separates specs from the broader *memory bank*: standing project context such as rules files, architecture notes and AGENTS.md. A memory bank applies to every task in the codebase; a spec applies only to the task or feature it describes.

Three depths of SDD

The same source distinguishes how long a spec lives.

Level	What happens to the spec	Who edits what	Today's reality
Spec-first	Written before the task and used to drive it; often discarded or left to rot afterwards	Humans and AI edit both spec and code	Nearly every SDD workflow starts here
Spec-anchored	Kept after the task and updated whenever behavior changes	Humans and AI edit both; tests keep them aligned	The aspiration of most teams adopting SDD

Level	What happens to the spec	Who edits what	Today's reality
Spec-as-source	The only file humans edit; code is generated and marked do-not-edit	Humans edit specs only	An aspiration; one InfoWorld reviewer doubts any current tool fully delivers it (InfoWorld)

A 2026 academic report by Deepak Babu Piskala uses the same three levels and offers guidance on when each is worth the effort (arXiv 2602.00180).

How SDD differs from its neighbours

Approach	What is written first	Source of truth	Main limitation
Vibe coding	A conversational prompt	The generated code	Intent lives only in a chat that is soon gone
Prompt / plan-mode workflow	A plan produced by the agent for one session	The code, plus a throwaway plan	Plan rarely survives the session
TDD	Failing tests	Code and tests	Tests capture examples, not the full intent or the why
BDD	Given/When/Then scenarios shared with business users	Scenarios plus code	Scenarios cover behavior, not constraints or architecture
Waterfall	A large requirements document, once	The document, in theory	Feedback arrives months later
SDD	A structured spec: intent, scope, constraints, interfaces, criteria, tests	The spec (at the anchored and source levels)	Overhead, drift, and agents that do not fully comply

SDD is often accused of being waterfall again, and the criticism is not baseless. The usual rebuttal is the feedback loop: an agent turns a spec into running code in minutes, so a wrong requirement shows up the same afternoon rather than after a release cycle (Alex

Cloudstar). Thoughtworks makes the same case that this time differs because the cycle is short. SDD also does not replace TDD or BDD. It sits above them: the acceptance criteria and test cases in a spec are what the tests are generated from.

Why the specification, not the prompt, is the source of truth

A source of truth is the artifact you consult when two things disagree and the one you fix last. In AI-assisted work, the candidates are the chat prompt, the generated code and the written spec. The case for the spec rests on what each can and cannot do.

Property	Prompt	Code	Specification
Lifespan	One session or one run	As long as the system	As long as the feature
Records the *why*	Sometimes, informally	Rarely	Yes, as intent
Reviewable before building	No	Only after it exists	Yes, in a pull request
Versioned and diffable	Seldom	Yes	Yes
Shared by humans and agents	No, it sits in one person's chat	Humans read it, agents re-infer it	Yes, both read the same file
Provides a test oracle	No	Only through its own tests	Yes, via acceptance criteria
Survives a change of model or tool	No	Yes	Yes

The arguments

1. **Prompts are ephemeral; specs persist.** A prompt exists inside a conversation. Close the window, switch agents, or let the context fill up and the intent is gone. A spec lives in the repository next to the code, so a new teammate or a fresh agent session starts from the same description.

2. **Code records what, not why.** Code tells you what the system does, including its accidents. It cannot tell you which behaviors were intended and which were bugs you tolerated. Without a statement of intent, an agent asked to refactor or extend must guess, and it often guesses by copying existing patterns, mistakes included.
3. **The bottleneck has moved.** When agents write the code, the limiting factor becomes how well you articulate intent. One practitioner guide puts it this way: as agents take on sustained autonomous work, the constraint shifts from how fast we write code to how well we state what we want (intent-driven.dev). A spec is where that articulation is done carefully rather than in passing.
4. **Ambiguity is cheapest to fix in the spec.** The most expensive error in AI-assisted work is building the wrong thing well (DEV Community). Catching a misunderstood requirement in a few lines of markdown costs a review comment. Catching it in a 2,000-line diff costs a rewrite.
5. **Verification needs an oracle.** To check that generated code is right, you need an independent statement of what "right" means. Acceptance criteria and test cases in the spec are that oracle. A prompt cannot play this role, because nobody agrees afterwards on what it promised.
6. **Models and agents are interchangeable; the contract is not.** Tools, models and agent frameworks change every few months. A spec written in plain markdown outlives all of them. Spec Kit, for example, is deliberately agent-agnostic and plugs into around thirty coding agents (InfoWorld).
7. **Specs are curated context.** Agents work from a finite context window and an uneven attention budget. A spec is the curated, structured subset of what the agent must know for this task, which is why Thoughtworks ties SDD closely to context engineering (Thoughtworks).
8. **Humans can review intent faster than code.** The aim is to shift human effort from reading generated code line by line to getting the specification right, and then confirming the code conforms to it. GitHub's framing is that maintaining software becomes a matter of evolving specifications, with code as the last mile (quoted by Böckeler).

What happens to the prompt

The prompt does not disappear; it gets thin. Instead of re-describing the feature in prose each time, the prompt points at the spec: "Implement task 3 from `specs/004-password-reset/tasks.md`, following `plan.md` and the project constitution." The durable content is in the files. The prompt is just the trigger.

An honest caveat

"Source of truth" is a claim about intent, and it is only as strong as the team's discipline. Three limits deserve to be stated plainly:

- At the spec-first level, the code becomes the real truth again after the first merge, because nobody updates the spec. Böckeler observed that Spec Kit creates a branch per spec, which suggests a spec lives for one change rather than for the feature's lifetime.
- Some Thoughtworks voices still argue that specs merely drive generation, as tests do in TDD, and that executable code remains the truth you must maintain.
- A spec that no longer matches the code is worse than no spec, because new contributors trust it (CODERCOPS).

The practical conclusion: the spec is the source of truth *for intent and acceptance*, the code is the source of truth *for what actually runs*, and tests and CI are the mechanism that keeps the two honest.

Anatomy of a good specification

A good spec answers six questions in order, each narrowing the space of acceptable implementations until an agent (or a human) has little room to build the wrong thing. No standard names these six parts; they are a synthesis of what the common tools and guides ask for. Kiro splits a spec into requirements, design and tasks files. Spec Kit moves from a constitution through specify, plan and tasks. Addy Osmani's guide to writing specs for agents centres on commands, testing, project structure, code style, git workflow and boundaries. The six components below cover the same ground in a tool-neutral way.

#	Component	Question it answers	Typical contents	What goes wrong without it
1	Intent	Why does this exist, and for whom?	Problem, user, desired outcome, success in plain language	Agent builds something plausible but pointless
2	Scope	What is included, and what is explicitly not?	In-scope behaviors, non-goals, deferred items	Feature creep and unrequested rewrites

#	Component	Question it answers	Typical contents	What goes wrong without it
3	Constraints	What must always hold?	Performance, security, compliance, tech stack, boundaries	Working code that is slow, unsafe or off-standard
4	Interfaces	How does it touch everything else?	APIs, schemas, events, UI contracts, data models, error shapes	Components that do not fit together
5	Acceptance criteria	What is true when it is done?	Testable, observable conditions, ideally one behavior each	Endless revision cycles, no definition of done
6	Test cases	What concrete examples prove it?	Inputs, expected outputs, edge and failure cases	Criteria nobody can actually run

Four rules that apply to every component

- **Separate the what and why from the how.** Intent, scope and acceptance criteria describe behavior. Design details come later, in a plan. Böckeler found the boundary hard to hold in practice, and a deep-dive on SDD warns that mixing the two makes the model treat your implementation hints as hard constraints.
- **Write to be tested.** Every statement should be one a test or a reviewer could confirm as true or false. "Fast" and "user-friendly" fail this test; "p95 latency under 200 ms" passes.
- **Size the spec to the change.** A one-line fix does not need six sections. Osmani warns against both extremes: a vague one-liner with no constraints, and a 500-line document that dictates every column name and is harder to maintain than the code (guide summary).
- **Keep it alive.** Store the spec in the repository, review it in pull requests, and update it in the same change as the code. Otherwise it decays into the misleading document described earlier.

The next six sections take each component in turn.

1. Intent: why this exists

Intent is the purpose of the work, stated so that someone with no context could judge whether a proposed solution serves it. It names the user or system affected, the problem, and the outcome that would count as success. Intent is what lets an agent make sensible choices in situations the spec never anticipated, because it can ask what the change is for.

What to include

- The problem in one or two sentences, from the user's or business's point of view.
- Who benefits, and how they will notice.
- The measurable outcome, if one exists (a rate, a time, a count).
- Any background that explains a surprising decision, such as an incident or a regulation.

Weak: "Add password reset."

Strong: "Users who forget their password currently have to contact support, which is a steady source of support tickets. Let them regain access themselves, safely, without revealing whether an email address has an account."

Common mistakes

- Writing the solution as the intent ("add a Reset button"). That hides the real goal and blocks better options.
- Leaving intent out because it feels obvious. It is obvious to you; the agent has only the text.
- Burying intent in a long document. Put it first, in a few lines.

Osmani recommends starting from a short high-level brief of what you are building and why, and letting the agent draft the detailed spec from it, so that you keep control of the direction while the model does the elaboration (Osmani).

2. Scope: what is in, and what is not

Scope draws the boundary around the work. It lists the behaviors that must be delivered and, just as importantly, names what will not be delivered. Agents are eager: given a loose boundary they will refactor neighbouring code, add features nobody asked for, or pull in new dependencies. A written boundary is the cheapest guard against that.

What to include

- In scope: the specific behaviors, user journeys or components to be built.

- Out of scope (non-goals): adjacent things a reader might assume are included.
- Deferred: items intentionally postponed, so nobody mistakes the omission for an oversight.
- Affected areas: which modules, services or directories may be changed, and which must not be.
- Assumptions and open questions, marked clearly rather than silently decided.

Weak: "Build a user management system." Osmani's guide uses this as the classic example of a spec with no constraints, no scope boundary and no acceptance criteria; you get something that functions but is probably not what you wanted.

Strong: "In scope: request a reset link by email, set a new password via a single-use link. Out of scope: SMS or authenticator-based recovery, admin-initiated resets, changes to the login page layout. Deferred: reset-attempt analytics dashboard."

Common mistakes

- Listing only what is in. Non-goals are where most scope disputes are settled in advance.
- Making scope so large that it cannot be implemented and reviewed as one unit. Break big features into separate specs, each small enough to complete and verify.
- Hiding unresolved questions. Spec Kit's workflow includes an explicit clarification step for exactly this reason; Kiro's quick-spec mode front-loads questions about scope, constraints and edge cases before generating anything.

3. Constraints: what must always hold

Constraints are the rules the solution must obey regardless of how it is built. They cover the qualities that functional descriptions leave out: performance, security, privacy, compliance, reliability, technology choices and working conventions. An agent that satisfies every requirement but ignores constraints produces code that passes a demo and fails in production.

What to include

- Non-functional targets with numbers: latency percentiles, throughput, availability, memory, cost.
- Security and privacy rules: what data is sensitive, what must never be logged, how secrets are handled.
- Technology and architecture limits: language and framework versions, approved libraries, patterns to follow, dependencies that may not be added.

- Working conventions: code style, branch and commit rules, test expectations.
- Boundaries on autonomy. Osmani suggests three tiers: things the agent always does without asking (such as running tests before committing), things it must ask about first (such as schema changes or new dependencies), and things it never does (such as committing secrets or editing vendored code). In the GitHub study of agent instruction files he cites, "never commit secrets" was the single most useful constraint.

Weak: "It should be secure and fast."

Strong: "Reset tokens are single-use, expire after 30 minutes, and are stored only as hashes. The request endpoint responds in under 300 ms at p95 and returns the same response whether or not the account exists. No token or email address may appear in logs. Use the existing mailer service; add no new dependencies."

Where constraints live. Project-wide constraints belong in the memory bank, not in every spec: Spec Kit calls it the constitution, Kiro calls the equivalent steering files (product, structure and tech documents), and many agents read an AGENTS.md. Feature specs then add only the constraints specific to that change. This avoids repeating the same rules in every file.

Common mistakes

- Piling on rules. Long instruction lists dilute attention and agents start to skip items. Keep the constraints that would actually change the output.
- Over-literal enforcement. Böckeler saw an agent go well beyond what was needed because it applied one constitution article too eagerly. State the reason behind a constraint so the agent applies it proportionately.
- Constraints with no check. Each one should map to a test, a lint rule or a CI gate, or it is only a hope.

4. Interfaces: how it connects to everything else

Interfaces are the contracts at the edges of the work: what goes in, what comes out, and in what shape. They are where independently built pieces either fit or do not. In SDD they are also the part of a spec that is most naturally precise and machine-checkable, which is why Tessl's example specs pin down the exposed API even while other details stay in prose (Böckeler).

What to include

- External interfaces: HTTP endpoints, CLI commands, events and messages, file formats.

- Internal interfaces: function or class signatures other modules will call, and module boundaries.
- Data models: entities, fields, types, nullability, ranges, relationships and migrations.
- Error contracts: error codes, messages and status codes for each failure, not only the happy path.
- Dependencies on other systems: what is called, what it guarantees, and what happens when it is down.
- UI contracts, where relevant: states a screen must support (loading, empty, error, success).

Prefer structured formats where they exist. OpenAPI, JSON Schema and Protocol Buffers remove structural ambiguity and can generate stubs and conformance tests. They do not capture everything. A recent paper on governing generated software notes that a requirement like "return the current user profile" still leaves open determinism, retries, external calls and latency, so schemas need to be paired with written behavior and constraints (arXiv 2605.12981).

Example (illustrative)

```
POST /v1/password-reset/request
  body:      { "email": string, RFC 5322, max 254 chars }
  202:       { "status": "accepted" }   // always, whether or not the account
exists
  400:       { "error": "invalid_email" }
  429:       { "error": "rate_limited", "retry_after_seconds": int }

POST /v1/password-reset/confirm
  body:      { "token": string, "new_password": string, 12-128 chars }
  200:       { "status": "password_updated" }
  400:       { "error": "invalid_or_expired_token" }
  422:       { "error": "weak_password", "rules_failed": [string] }
```

Common mistakes

- Specifying only the success response. Agents then invent error behavior, and different pieces invent different behavior.
- Letting the agent redefine an interface mid-task. Mark published interfaces as fixed, and require an explicit spec change before they move.
- Duplicating the interface in prose and in a schema. Keep one authoritative version and link to it, or the two will drift.

5. Acceptance criteria: what "done" means

Acceptance criteria are the observable conditions that must be true for the work to be accepted. They turn intent and scope into a checklist that a person, a test or an agent can evaluate. They are the hinge of the spec: everything above them says what is wanted, everything below them proves it.

Properties of a good criterion

- **Testable.** A test or a reviewer can decide yes or no.
- **Measurable.** Numbers, not adjectives.
- **Atomic.** One behavior per criterion, so a failure points at one thing.
- **Unambiguous.** Two readers would write the same test from it.
- **Complete across paths.** Covers normal flow, errors, empty and boundary cases, not only the happy path.

Writing format. Two formats dominate. Given/When/Then comes from BDD. EARS (Easy Approach to Requirements Syntax), introduced in a 2009 requirements-engineering paper, constrains each requirement to one of a few sentence templates. Kiro adopted EARS for the acceptance criteria in its requirements files, and the structure suits LLMs because it is regular enough to parse and plain enough to read (Wikipedia: EARS).

EARS pattern	Template	Example
Ubiquitous	The system shall *response*	The system shall store reset tokens only as hashes
Event-driven	When *trigger*, the system shall *response*	When a valid reset link is submitted with a compliant password, the system shall update the password and invalidate the token
State-driven	While *state*, the system shall *response*	While an account is locked, the system shall not send reset emails
Optional feature	Where *feature is present*, the system shall *response*	Where multi-factor authentication is enabled, the system shall require the second factor after reset
Unwanted behavior	If *condition*, then the system shall *response*	If the token is expired, then the system shall reject the request with invalid_or_expired_token

Practitioner guides add a few conventions: use *shall* for mandatory behavior and avoid *should*, *may* and *will*, which blur whether something is required.

Weak: "Reset works correctly and is easy to use."

Strong: "When a user requests a reset, the system shall send one email containing a link that expires in 30 minutes. If a second request arrives within 60 seconds for the same address, then the system shall respond 429 and send nothing."

Common mistakes

- Vague adjectives ("robust", "gracefully", "intuitive"). Böckeler's small-bug experiment produced criteria such as handling edge cases gracefully, which no test can check.
- Too many criteria for a small change. The same experiment turned one bug into four user stories and sixteen criteria. Match the number of criteria to the risk.
- Criteria that restate the implementation ("uses bcrypt") instead of the outcome. Put implementation choices in constraints or the plan.
- No link to tests. Every criterion should map to at least one test case, and every test case to a criterion.

6. Test cases: concrete proof

Acceptance criteria say what must be true; test cases show it with specific inputs and expected outputs. They remove the last layer of ambiguity, because an example cannot be read two ways. They also give the agent a target it can run against its own work: agents that execute tests as part of their loop catch more problems before handing work back, and writing testing expectations into the spec ensures tests are part of the plan (guide summary).

What to include

- Happy-path cases for each acceptance criterion.
- Boundary cases: minimum and maximum lengths, exact expiry times, empty and null values.
- Negative and failure cases: invalid input, dependency outages, permission failures.
- Abuse and security cases: replay, enumeration, injection, rate-limit evasion.
- Regression cases for behavior that must not change, which is especially important when the agent edits existing code.
- Non-functional checks where the constraints demand them (a latency or load test).

Keep a traceability link. Each case should reference the criterion it proves, and each criterion should have at least one case. Kiro's task lists do this by tagging tasks with requirement numbers, and its design files tie correctness properties back to specific requirements (Mastering Kiro guide). A gap in this mapping is a gap in the spec.

Example (illustrative)

ID	Proves	Scenario	Setup / input	Expected result
T1	AC-1	Reset for existing account	Request reset for a registered email	202; one email sent; link valid for 30 minutes
T2	AC-2	No account enumeration	Request reset for an unregistered email	202 with identical body; no email sent
T3	AC-3	Expired token	Submit a token 31 minutes old	400 invalid_or_expired_token; password unchanged
T4	AC-4	Single use	Submit the same valid token twice	First 200; second 400 invalid_or_expired_token
T5	AC-5	Rate limit	Two requests for one address within 60 s	Second returns 429 with retry_after_seconds; no second email
T6	AC-6	Weak password	New password of 8 characters	422 weak_password; token still valid
T7	C-3	No sensitive data in logs	Run T1 and inspect logs	Neither token nor email address appears

Practices that make test cases work with agents

- Write or approve test cases before implementation, and treat them as the contract. Letting an agent write both the code and the only tests that judge it invites tests that merely confirm what the code happens to do.
- Instruct the agent not to weaken or delete tests to make them pass, and review any change to them as carefully as a change to the spec.
- Remember what approval means. An approved spec does not prove the requirements were right or the tests sufficient; it shows only that the implementation matches what you approved (Verdent on Kiro).

Common mistakes

- Only happy-path cases, which let the agent pass while failing in the situations that matter.
- Examples with magic values and no explanation. Say why the boundary is where it is.
- Test cases that duplicate acceptance criteria word for word. A case should add a concrete input and output.

Worked example: all six parts in one compact spec

The spec below pulls the earlier fragments together for one feature. The identifiers (AC-1, C-3, T1) match the test table in the previous section, so you can see the traceability. It is a template to adapt, and the figures are illustrative.

```
# Spec: Self-service password reset

## 1. Intent
Users who forget their password currently have to contact support.
Let them regain access themselves, safely, without revealing whether
an email address has an account.

## 2. Scope
In scope:     request a reset link by email; set a new password via a
              single-use link.
Out of scope: SMS or authenticator recovery; admin-initiated resets;
              login page layout changes.
Deferred:     reset analytics dashboard.
May change:   /auth/*, /mailer/templates/reset.*
Must not change: session handling, existing login endpoint.

## 3. Constraints
C-1  Tokens are single-use, expire after 30 minutes, stored only as hashes.
C-2  Request endpoint: p95 under 300 ms; response identical whether or
     not the account exists.
C-3  No token or email address in any log line.
C-4  Use the existing mailer service; add no new dependencies.
Ask first: any schema change.   Never: touch secrets or CI config.

## 4. Interfaces
POST /v1/password-reset/request   -> 202 | 400 | 429
POST /v1/password-reset/confirm   -> 200 | 400 | 422
```

```
Full request and response shapes: see openapi/password-reset.yaml
New table: password_reset_tokens(id, user_id, token_hash, expires_at,
used_at)

## 5. Acceptance criteria (EARS)
AC-1  When a registered address requests a reset, the system shall send one
      email with a link valid for 30 minutes.
AC-2  When an unregistered address requests a reset, the system shall return
      the same 202 response and send no email.
AC-3  If a token is expired, then the system shall reject it with
      invalid_or_expired_token and leave the password unchanged.
AC-4  When a token has been used, the system shall reject any reuse.
AC-5  If two requests arrive for one address within 60 seconds, then the
      system shall return 429 and send no second email.
AC-6  If the new password is under 12 characters, then the system shall
      return 422 weak_password and keep the token valid.

## 6. Test cases
T1-T7, each mapped to a criterion or constraint: see
tests/password_reset.spec.md

## Open questions
- Should a successful reset sign out other active sessions?
```

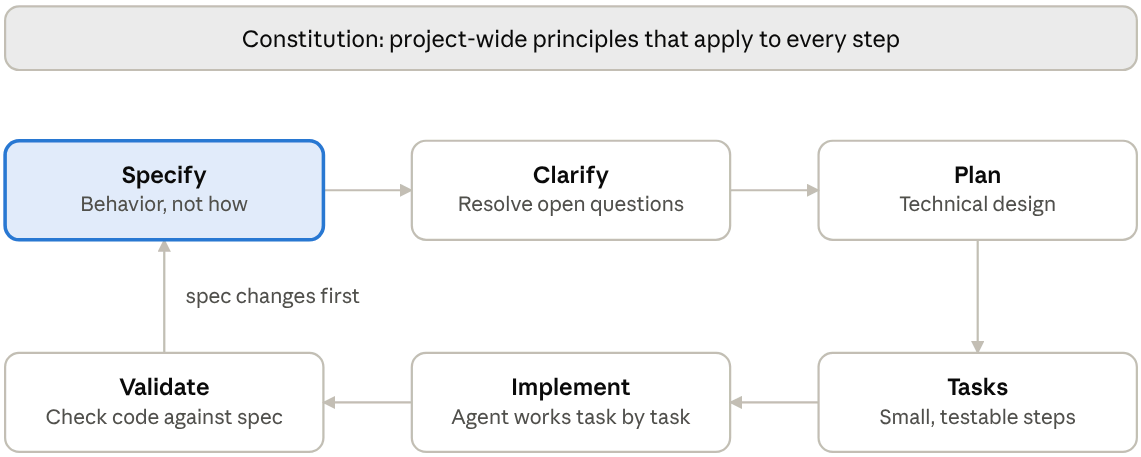
The prompt that goes with this spec is short: "Read specs/004-password-reset/spec.md. Write the plan first and stop for review. After approval, implement the tasks one at a time, running the tests after each, and do not edit the tests." The detail lives in the file, not the message.

Note the open question at the end. A spec that states what it does not yet know is more useful than one that quietly guesses.

The SDD workflow and the tooling landscape

Most SDD tools converge on one loop: write the spec, refine it, plan the technical approach, break the plan into tasks, let the agent implement, then check the result against the spec. Spec Kit names its opening steps constitution, specify, plan and tasks. Kiro uses three files: requirements, design and tasks. Both ask you to review at each stage (Böckeler).

Every step builds on the spec, and changes loop back to it



You review and refine the output of each step before the next one starts.

SDD workflow · 6 steps, 1 feedback loop

The loop has one rule: a change in behavior goes into the spec first and the code second. That rule is what keeps a spec anchored instead of abandoned.

Tools at a glance

The tools differ more than their shared label suggests. They change quickly, so check current documentation before standardizing on one.

Tool	What it is	How a spec is structured	Level it targets
GitHub Spec Kit	Open-source, MIT-licensed CLI and templates that add an SDD workflow to the coding agent you already use; GitHub describes it as an experiment (review)	Constitution, then spec, plan and tasks files per feature, with checklists as a definition of done	Spec-first in practice; aims at anchored. Böckeler found it the heaviest of three, with a branch and many files per spec
AWS Kiro	Spec-driven IDE, also available as a CLI and web agent	requirements.md (user stories with EARS acceptance criteria), design.md, tasks.md, plus steering files for project context	Mostly spec-first; the lightest of the three. Offers bugfix specs and a quicker spec mode

Tool	What it is	How a spec is structured	Level it targets
Tessl	Spec registry and framework	Specs map one-to-one to code files; generated files are marked do-not-edit; interfaces are defined in the spec	Aspires to spec-anchored and spec-as-source; non-deterministic regeneration was observed in early trials
OpenSpec, BMAD-METHOD, Task Master	Lighter CLIs and methodologies layered on an existing agent (comparison)	Change-based specs (OpenSpec); a full agile-style agent method (BMAD)	Spec-first
Claude Code, Cursor and similar agents	General agents with no fixed SDD method	You supply the workflow: spec files, plan mode, and rules or AGENTS.md (Thoughtworks)	Whatever you build

Choosing between them

This is a judgment from the sources above rather than a benchmark, so treat it as a starting point.

- **Want something light inside an editor, with a clear requirements-design-tasks shape?** Kiro's three-file model is the easiest to learn.
- **Want to keep your current agent and customize templates?** Spec Kit is agent-agnostic and every artifact lands in your repository, but plan for review load.
- **Want to experiment with specs as the only edited artifact?** Tessl is the clearest attempt, with the least maturity.
- **Want no new tool at all?** A `specs/` folder, a template with the six components from this report, and an AGENTS.md covers most of the benefit.

Risks, criticisms and when not to use SDD

SDD has real costs, and the more careful sources say so. Treat these as design constraints on how you adopt it.

Risk	What it looks like	Mitigation
Sledgehammer for small changes	Böckeler's one small bug fix became four user stories and sixteen acceptance criteria in Kiro (source)	Use a light path (one paragraph plus two or three criteria) for small work, and skip the spec for trivial edits
Review overload	Many repetitive markdown files that overlap each other and the code; Böckeler said she would sooner review code	Keep specs short, avoid duplicating content across files, and review the spec as a pull request
False sense of control	Agents skip instructions, duplicate existing code, or over-apply a rule even with detailed specs; one practitioner reports agents completing about 80 to 90 percent of what was specified (Sibylline)	Keep reviewing output, run tests after every task, and check that each acceptance criterion is actually met
Spec drift	The code changes and the spec does not, until the spec describes a system that no longer exists (CODERCOPS)	Update spec and code in the same pull request; decide which file wins when they disagree
Non-deterministic regeneration	The same spec produces different code on different runs, which undermines spec-as-source (deep dive)	Pin interfaces, keep tests as the arbiter, and avoid do-not-edit generation until it is repeatable
Blurry what/how boundary	Functional specs fill with implementation detail, which the model then treats as binding	Keep behavior in the spec and technical choices in the plan or constraints
Waterfall relapse	Long up-front documents and late feedback	Write small specs, build one slice at a time, and use the fast feedback the agent gives you
Semantic gap	Prose and schemas leave behavior unstated, such as retries, determinism and latency (arXiv 2605.12981)	State non-functional constraints with numbers and back them with executable checks

Böckeler also draws a parallel with model-driven development. Its models were the specs, and it never took off for business applications because it sat at an awkward level of abstraction. Spec-as-source with LLMs could inherit both the inflexibility of that approach and the non-determinism of the models. She stresses that the point is to learn from the past, not to bring it back.

A further critique is that SDD is old wine in new bottles, that the value lies in the thinking done while writing the spec rather than in any tool (summary). That is a fair reading and not a rebuttal of SDD; it is a reason to invest in the writing.

When SDD pays off

- Features with real ambiguity, several moving parts or multiple contributors, human or agent.
- Work where being wrong is expensive: security, payments, compliance, data pipelines.
- Long-lived systems where intent has to survive staff and tooling changes.
- Parallel agent work, which needs a shared contract.

When to skip it or go light

- One-line fixes, throwaway prototypes and exploratory spikes where you do not yet know what you want.
- Very small teams in fast-changing situations, where specification overhead can eat a disproportionate share of the time (Augment guide).
- Problems so unclear that the first job is research and stakeholder discussion, not a spec.

Adoption checklist and spec review rubric

Starting small

The checklist below is a suggested path, not a standard. Tick items off as you go.

- [ ] Pick one medium-sized feature with real ambiguity as a pilot, not a one-line fix and not a whole product.
- [ ] Create a `specs/<feature>/` folder in the repository and a one-page template with the six components.

- [ ] Write project-wide rules once (constitution, steering files or AGENTS.md): stack, conventions, boundaries, and what the agent must never touch.
- [ ] Write intent and scope first, then have the agent draft the rest from your brief, and edit the draft yourself.
- [ ] Turn every acceptance criterion into at least one test case before implementation starts.
- [ ] Ask the agent for a plan and tasks, and review both before any code is written.
- [ ] Run tests after each task, and forbid edits to tests without review.
- [ ] Update the spec in the same pull request as any behavior change.
- [ ] After the pilot, record where the spec helped, where it was overkill, and which sections nobody read. Trim the template accordingly.
- [ ] Agree which level you are aiming for (spec-first, anchored or as-source) and which file wins when spec and code disagree.

Spec review rubric

Use this in pull-request review of the spec itself, before the agent starts.

Component	Passes when	Red flag
Intent	A stranger can say who benefits and what success looks like	It restates the solution ("add a button")
Scope	Non-goals are listed and the files that may change are named	Only inclusions; no boundary
Constraints	Each has a number or a rule and a way to check it	Adjectives such as secure, fast, scalable
Interfaces	Success and error shapes are defined; one authoritative source	Happy path only; duplicated definitions
Acceptance criteria	Atomic, testable, covers normal, error and boundary paths	Vague words; one criterion bundling several behaviors
Test cases	Each maps to a criterion and has concrete inputs and outputs	Only happy-path cases; no traceability
Whole spec	Short enough to read in one sitting, with open questions stated	Longer than the code it describes; guesses hidden as facts

One closing test comes from practitioners: if a capable stranger read this spec, would they build the thing you have in mind? If not, the gap is in the spec, not the model.

Sources

Gathered 29 October 2026. Fowler's article was read in full; the other pages were read as search excerpts, so the tooling details in them may have moved since publication.

- Birgitta Böckeler, Understanding Spec-Driven-Development: Kiro, spec-kit, and Tessl, martinfowler.com, 15 Oct 2025
- Thoughtworks, Spec-driven development: unpacking one of 2025's key new AI-assisted engineering practices
- Deepak Babu Piskala, Spec-Driven Development: From Code to Contract in the Age of AI Coding Assistants, arXiv, 2026
- Addy Osmani, How to write a good spec for AI agents, 13 Jan 2026 (also O'Reilly Radar)
- Martin Heller, 4 cutting-edge tools for spec-driven development, InfoWorld, 15 May 2026
- GitHub, spec-kit repository; Visual Studio Magazine, GitHub Open Sources Kit for Spec-Driven AI Development
- Kiro, From chat to specs: a deep dive; Verdent, Kiro Specs: Requirements to Code
- Wikipedia, Easy Approach to Requirements Syntax
- CodeMySpec, GitHub Spec Kit review and Tessl review
- Particula, Spec-driven development tools compared
- Critiques: Sibylline Software; Alex Cloudstar; CODERCOPS; deep dive on Medium; Augment Code guide
- Other: Allstacks guide; intent-driven.dev; Prefactor glossary; Protocol-Driven Development, arXiv 2605.12981; DEV Community overview; DEV Community on SDD costs; Mastering Kiro; spec-writing guide summary